Where Scoped Retry Goes Wrong: Heap and Trail Graphs for Two Failing Programs

Rice project notes

Abstract

`backtracking-optimizations.md`'s optimization #3 (scoped backtracking), generalized so that every generated `_hnf` function calls `SCOPED_CUTOFF()` on entry and `SCOPED_RETRY(...)` on a failing case, silently gives wrong answers on real programs. This note draws the heap graph and the backtracking trail, step by step, for two small programs that expose it, and pins the failures on two independent defects. Both are the same kind of mistake: after a scoped `undo`, the trail no longer contains a record for every heap cell that has been overwritten since the choice point it is supposed to protect. Sharing enters in two ways: graph reduction overwrites *shared* cells in place (which is the only reason there is a trail at all), and in the first example a *shared choice* ties a failure deep inside one case to a decision made by another. A third ingredient, control passing along heap edges instead of the C stack, is what makes a stack-height cutoff the wrong notion of scope in the second. Experiments with both defects patched in scratch copies produce correct output on all three programs tried; that shows the defects are sufficient to explain these failures, not that the optimization is now sound (Section 4).

1 Reading the figures

1.1 What the runtime does, in four sentences

1. **Evaluation overwrites cells in place.** Reducing a node replaces its contents with its result (a forward to another cell, a constructor, `FAIL`). Every parent that shares the node sees the result. This is what makes evaluation lazy *and* shared.
2. **The trail is a log of those overwrites.** A frame is a pair cell $\mapsto$ pre-image, plus a *choice* bit. `choose` pushes a frame for the choice cell (choice bit set if another alternative remains); `save_copy(root)` pushes one for a node about to be overwritten, but only when the node's `nondet` flag says it depended on some choice.
3. **`undo(c)` pops frames, restoring each cell, until it pops a choice frame or the stack height reaches c.** It returns whether it stopped on a choice frame. Global backtracking is `undo(0)`.
4. **Scoped retry** records `_cutoff = size(bt_stack)` when a generated function starts. When a case fails it calls `undo(_cutoff)`: "retry against any choice made *since I started*." The assumption is that the frames above `_cutoff` are exactly the overwrites made on behalf of this function's own computation.

The relevant code (`runtime/runtime.c`, `runtime/runtime.h`):

```
#define SCOPED_CUTOFF()  int _cutoff = size(bt_stack)
#define SCOPED_RETRY(scrutinee_var, orig_var) \
  if (undo(_cutoff)) { scrutinee_var = orig_var; continue; }

bool undo(int cutoff) {
    if(bt_stack->size == 0) return false;
```

```
    FieldPair* frame;
    do {                                        // always pops at least once
        frame = pop(bt_stack);
        if(frame->lhs.n != NULL) memcpy(frame->lhs.n, frame->rhs.n, sizeof(Node));
    } while(!frame->choice && size(bt_stack) > cutoff);
    return frame->choice;
}
```

1.2 Notation

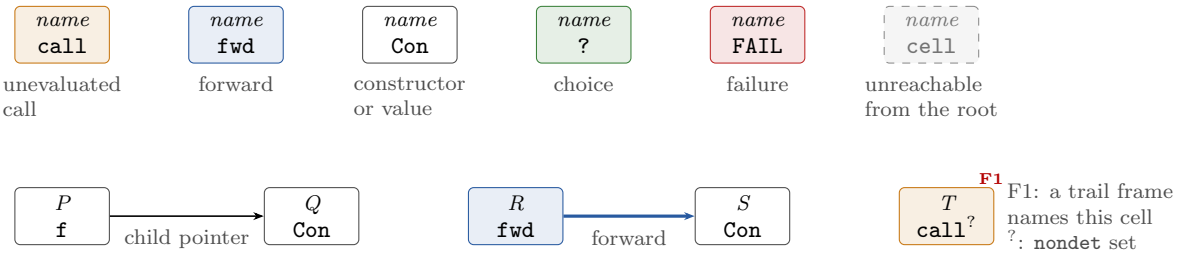


Figure 1: Cell colours and marks. Children are drawn left to right in source order (Rice's debug printer prints them right to left; the traces the figures were taken from are reversed accordingly).

A cell that has been overwritten *loses its old children*: after `save_copy` and a rewrite the only remaining reference to the old form is the trail frame's pre-image, which is written inside the trail box, not drawn as a heap edge. Trail frames are named **F0**, **F1**, ... (bottom to top, in the order pushed), and a red rule in the trail box marks the `_cutoff` of the function about to call `undo`: *everything above the rule is in that function's scope.*

2 Example 1: a shared choice, and a lost solution

```
g :: Bool -> Bool
g True  = failed
g False = False

f :: Bool -> Bool -> Int
f True  y = case y of False -> 1
f False y = case y of False -> 2

main :: Int
main = let c = True ? False in f c (g c)
```

The choice `c` is used twice: as the scrutinee of `f`, and inside the argument `g c`. The two uses must agree. The only consistent solution is `c = False`: then `g c = False` and `f False False = 2`. With `c = True`, `g c` fails. Scoped retry off gives `FAIL, 2`; the default build gives `FAIL, FAIL`, i.e. the solution is lost.

After inlining, the compiler produces one cell per call. I call them **F** (`main_CASE5`, the outer case of `f`) and **G** (`main_CASE3`, the body of `g c`); the True branch of `f` is `main_CASE7`. Each case branch is its own C function with its own `_cutoff`.

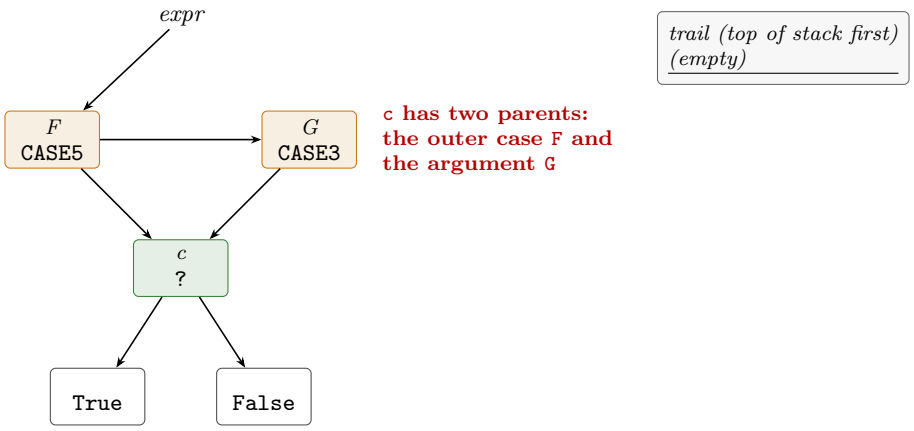


Figure 2: Before anything is evaluated. `F = f c G`, `G = g c`, and both share the choice cell `c`. (`c` is really a let-thunk that becomes this choice node when first forced; shown already forced.)

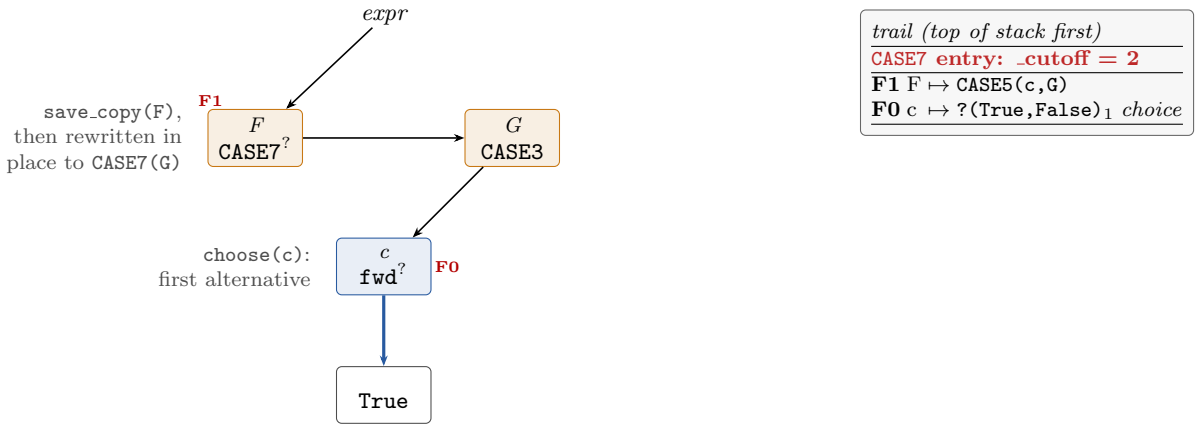


Figure 3: `f`'s outer case dispatches on `c`. `F0` is the choice frame (`c`'s other alternative is still pending). `F` is then overwritten in place: it forgets `c` and now holds only `G`. `F1` is its old form. `CASE7`, the True branch of `f`, starts with `_cutoff = 2`.

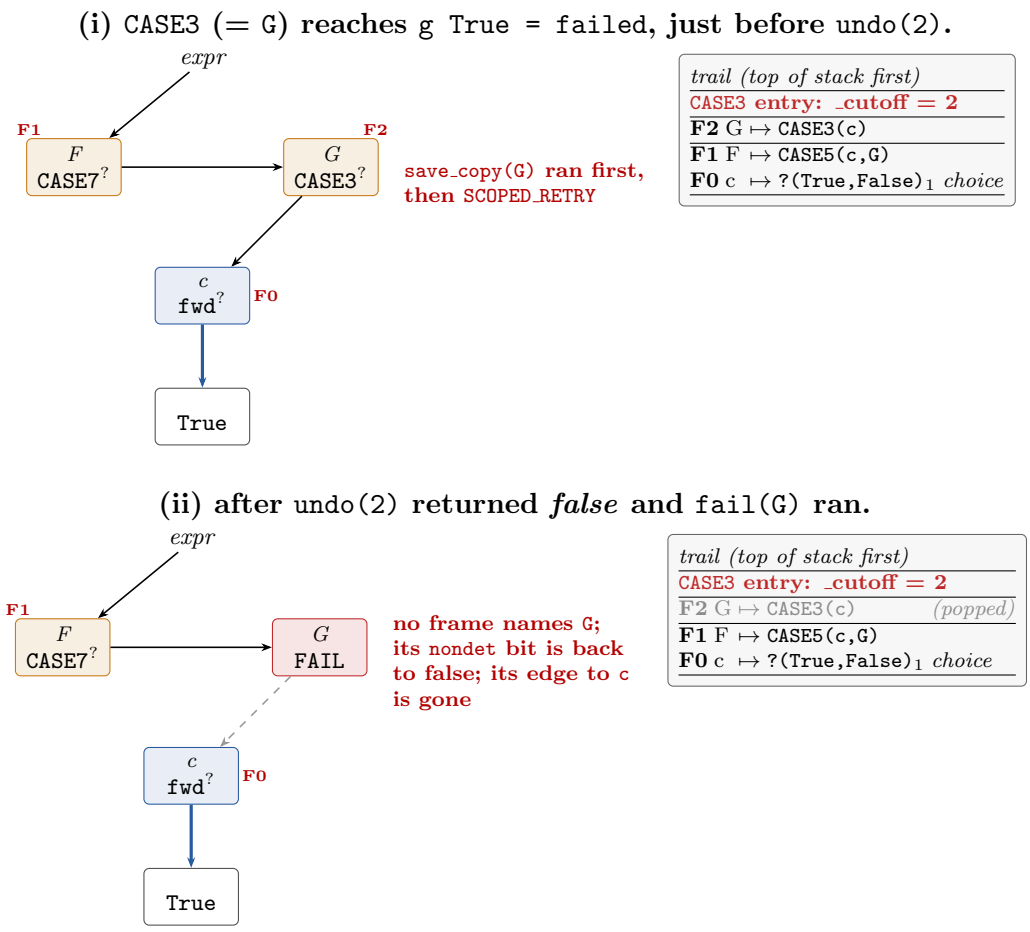


Figure 4: Defect 2. `showConsBranch` emits `save_copy(root)` as the first thing in a constructor branch, so by the time the branch body reaches an unhandled case (`IExempt`) and calls `SCOPED_RETRY`, the scope already contains the frame `F2` that `save_copy` just pushed. `undo(2)` finds no choice frame, so it returns false, but only after popping `F2`: `G` is restored to its raw form and the frame is gone. The next line, `fail(G)`, then overwrites `G` with no trail record at all. Restoring `G` also cleared its `nondet` bit (the pre-image was saved before the bit was set), so nothing downstream can tell this `FAIL` came from a choice. (The `FAIL_TAG` path is emitted in the correct order: retry first, *then* `save_copy`, then `fail`.)

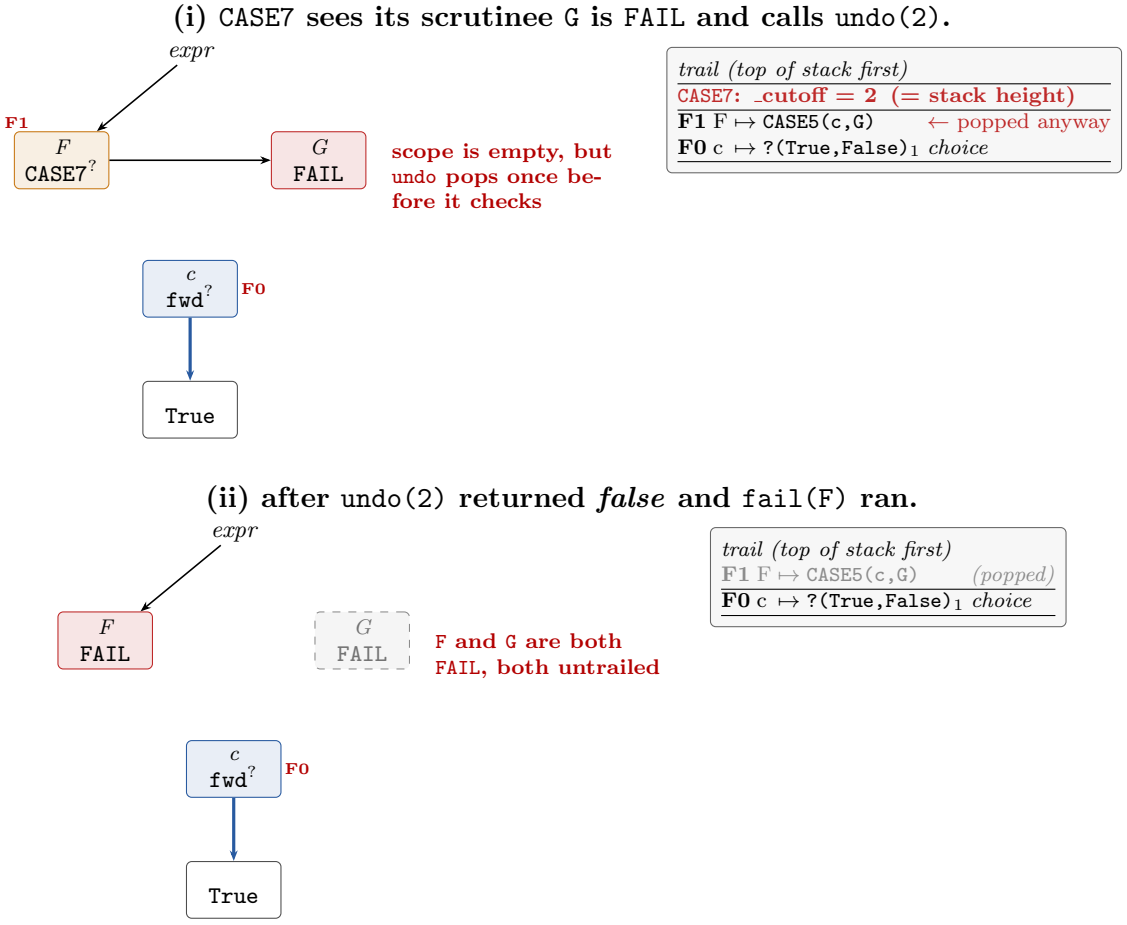


Figure 5: Defect 1. `CASE7`'s scope is empty: `F1` was pushed by `CASE5` *before* it dispatched into `CASE7`, so it sits below `CASE7`'s cutoff. But `undo` is a `do`/`while` and always pops at least one frame, so it pops `F1` and restores `F` to `CASE5(c,G)`. Then `fail(F)` overwrites `F` with `FAIL`; because `G`'s `nondet` bit was lost in Figure 4, `CASE7`'s own `nondet` is false and it does not re-save `F`. (Had the popped frame been a *choice* frame, `undo` would have returned true, and a callee would have flipped a choice belonging to its ancestor.)

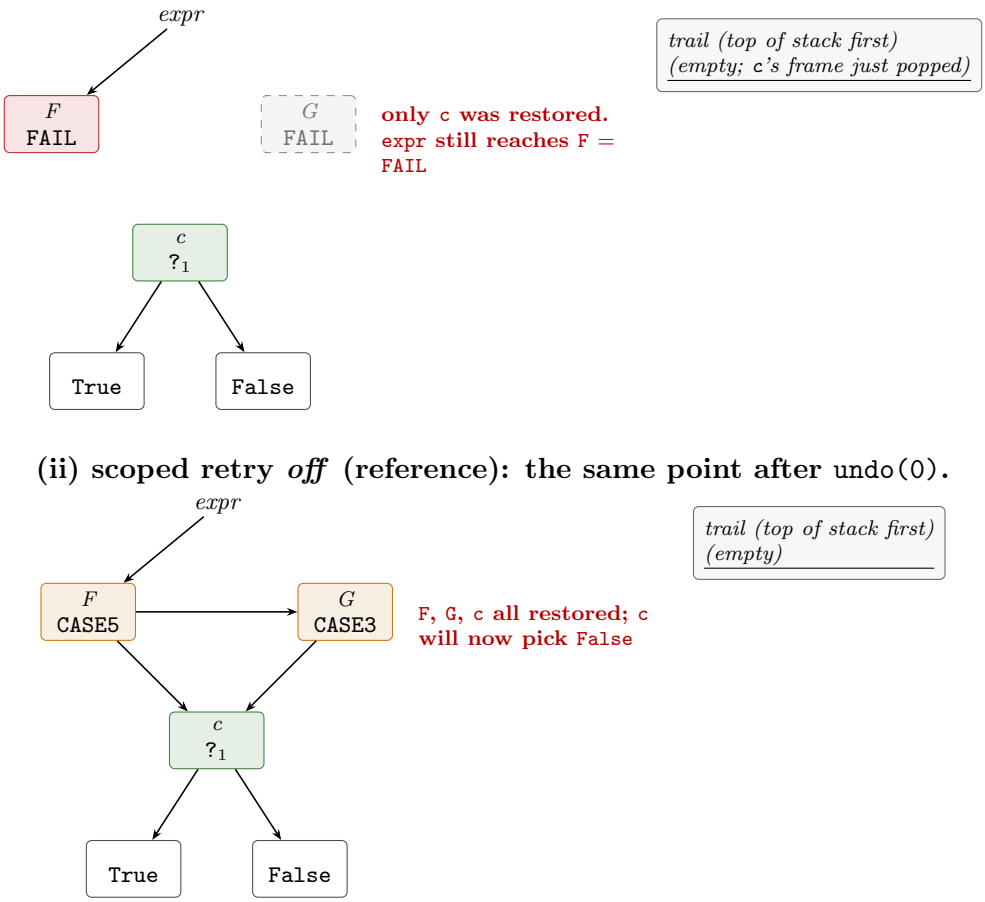


Figure 6: The consequence. (i) shows Figure 2 minus `F` and `G`: only the cell with a trail frame (`c`) is restored. The driver re-normalizes `expr`, follows it to `F`, and finds `FAIL` without ever re-evaluating anything, so `c = False` is never tried. (ii) is what backtracking must produce. With scoped retry off the trail at failure time was, top first: `F` ↦ `CASE7(G)`, `G` ↦ `CASE3(c)`, `F` ↦ `CASE5(c,G)`, `c` ↦ `?` (choice); popping all four restores all three cells, and the shared choice `c` is set to flip. The next run takes `c = False`, `g False = False`, `f False False = 2`.

3 Example 2: control passes through a heap edge

This is the program from `debug_repros/cent_tiny.curry`: `eval (mkExp2 3 0)`, where `mkExp2` builds a chain of four alternatives `e1 :+ e2 ? e1 :- e2 ? e1 :* e2 ? e1 :/ e2` and `eval (x :/ y)` has an incomplete `case` (no `False` branch when `y' > 0 && cond` is false). Only the last alternative, `3 :/ 0`, fails. With scoped retry off the run prints `3, 3, 0, FAIL`; the default build prints `3, 3, 0` and then the raw, unevaluated `eval(:/(Num(0), Num(3)))`. (That is `eval (Num 3 :/ Num 0)`; the debug printer lists children right to left.)

Here the failure happens inside a *different function* from the one that made the choice, and nothing on the C stack connects them.

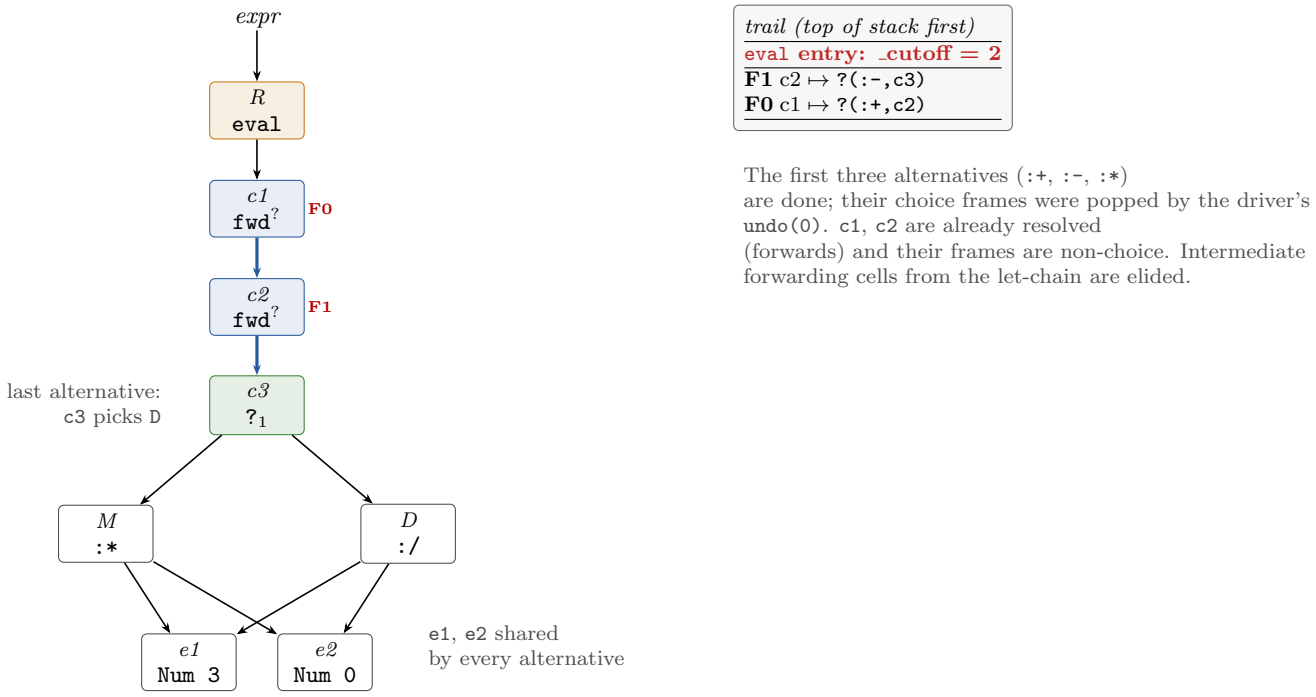


Figure 7: Start of the fourth and last attempt. `R` is the top-level `eval` node, shared by the driver's `expr` and, later, by trail frames. `c1`, `c2` were resolved to their right alternatives on earlier attempts. `c3` is the last live choice; `eval` is about to force it.

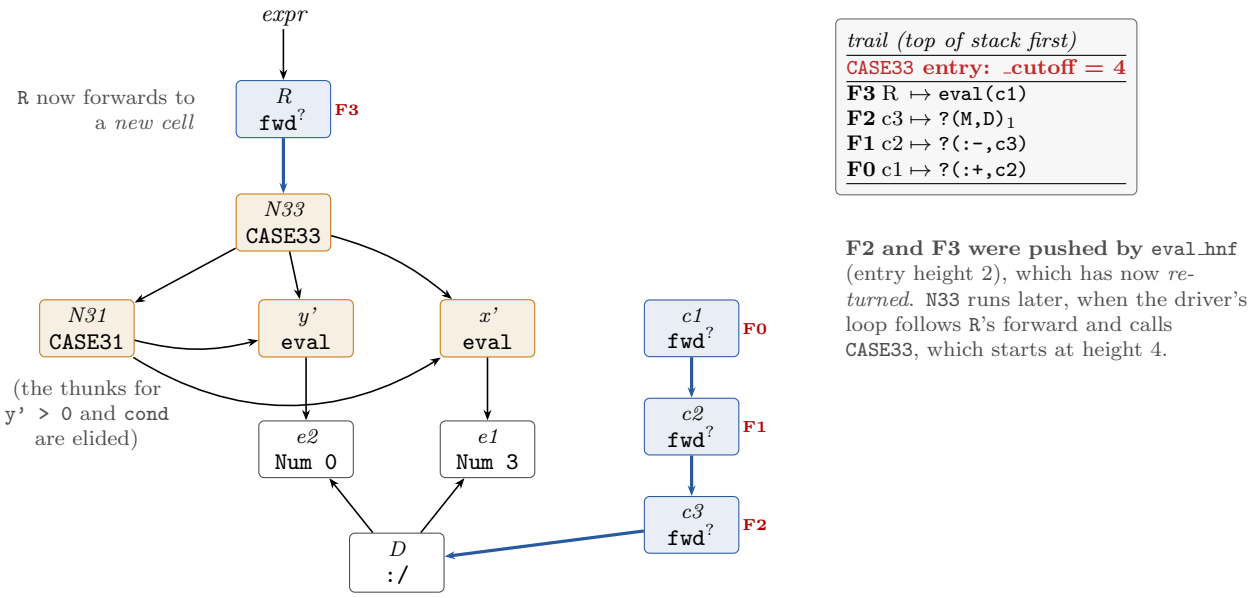


Figure 8: `eval_hnf(R)` has finished. It pushed `F2` (`c3` narrowed to `D`) and `F3` (`R`'s pre-image), rewrote `R` through a chain of tail calls, and ended with `collapse(R, N33)`: `R` now *forwards* to a fresh cell `N33 = CASE33(N31, y', x')`. The remaining work (`N33`) lives in a different cell from the one whose pre-image the trail holds. `y'` and `x'` are shared by `N33` and by the elided thunks under `N31`. The driver then calls `CASE33_hnf(N33)`, which takes its own `_cutoff = size(bt_stack) = 4`: the top of the stack, above `F3`.

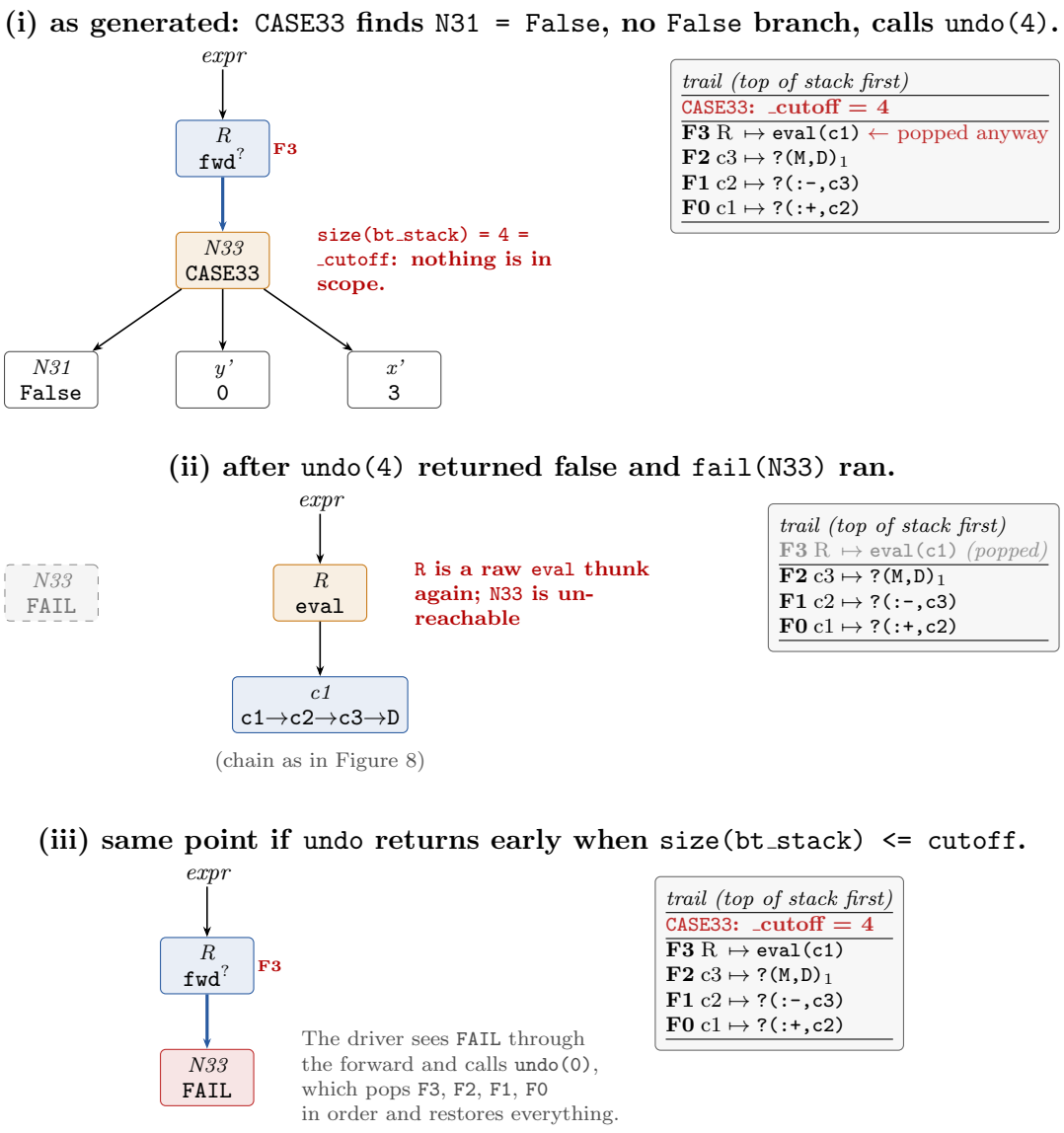


Figure 9: Defect 1 again, from the other side. `F3` was pushed by `eval_hnf` before it handed off to `N33`, so it sits *below* `CASE33`'s cutoff: `CASE33`'s scope is empty. `undo(4)` pops it anyway, restoring `R` to the raw thunk `eval(c1)`, which severs `R`'s forward to `N33`. The frame was not a choice frame, so `undo` returns false and `CASE33` falls through to `fail(N33)`, which marks a cell nobody points to any more. The driver looks at `expr`, finds a non-`FAIL` node, and prints it as a solution. (iii) With the guard, `CASE33` leaves the stack alone and falls through to today's unconditional `fail`. The driver sees `FAIL` through the forward and `undo(0)` pops `F3`, `F2`, `F1`, `F0` in order, restoring everything: the same behavior as with scoped retry off.

4 What the two failures have in common

4.1 The invariant that breaks

Undo is only correct if, at every moment, the trail contains a frame for every cell that has been overwritten since the choice point being backtracked to. Global `undo(0)` preserves this trivially: it empties the log, so nothing is left to disagree with. *A scoped undo removes a suffix of the log and then keeps executing*, so the invariant has to hold for the frames it *leaves* as well as the ones it removes. Both defects break it:

Defect 1: `undo` pops a frame outside its scope.
The `do/while` pops once before testing `size > cutoff`. If the scope is empty (`size == cutoff`) it removes a frame that belongs to an ancestor computation and restores that ancestor's cell. Figures 5 and 9. In Example 2 the ancestor is `eval_hnf`: the trail is address-keyed and `R`'s pre-image is just "a frame", with nothing to say it belongs to a computation whose continuation lives in another cell.

Defect 2: a failed retry leaves `root` mutated without a frame.
In an `IExempt` branch the emitter (`ToC.curry`, `showConsBranch` + `showStmt ctx n IExempt`) writes `save_copy(root); SCOPED_RETRY; fail(root)`. An unsuccessful retry pops the frame `save_copy` just pushed, so `fail(root)` mutates `root` unlogged, and restoring the pre-image also resets `root`'s `nondet` bit to its earlier value (false in Figure 4). The `FAIL_TAG` path is emitted in the right order and does not have this problem.

4.2 Where sharing comes in

- **Why there is a log at all.** Every parent of a shared cell sees its in-place update. `R` is shared between the driver and the trail; `G` and `c` are shared between two branches of the same call. Undo has to restore cells *by address*, whichever parent asks.
- **Why the log cannot tell whose entry it is.** Frames are `cell` $\mapsto$ `pre-image`. They carry no record of which function pushed them. `_cutoff` is a stack height, an approximation of "mine". It is exact only while a function's whole computation stays inside one C activation. `TAIL_DISPATCH` and `collapse` break that: control moves along a heap edge (`R` $\rightarrow$ `N33`) or into a sibling function (`CASE7`), and the callee's height sits *above* its logical parent's frames.
- **Why a shared choice makes the failure non-local.** In Example 1 the choice `c` is made by `F`'s dispatch, but the failure it causes is found inside `G`. `c`'s frame is below `G`'s cutoff, so `G` cannot retry it. The only correct move is to fail *and say why*: leave a frame that restores `G` and keep `G`'s `nondet` bit, so that `F` logs itself and global backtracking can revisit `c`. Defect 2 destroys exactly that information.

4.3 What I ran

Every variant below was built in a scratch directory from the current tree: `-DOPT_SCOPED_BACKTRACK=0` for "off"; the tree as is for "on"; *guard* is a one-line change to a scratch copy of `undo` (return false if `size(bt_stack) <= cutoff`); *re-save* is a text patch to the generated C (and to `Prelude.c`) that inserts `if(nondet) save_copy(root);` between `SCOPED_RETRY` and `fail(root)` in the `IExempt` path. No source file in the repository was edited; the only additions are this note and its PDF.

	off	on (default)	guard only	re-save only	guard + re-save
Example 1 (`nested`)	ok	**lost 2**	**lost 2**	ok	ok
Example 2 (`cent_tiny`)	ok	**prints raw `eval(...)`**	ok	—	ok
`cent` (528 solutions)	ok	**1133 distinct**	ok	—	ok

Table 1: "ok" means the real solutions match scoped retry off (for `cent`: exactly the 528 lines of `cent.expected`, after dropping `SOLUTION: FAIL` lines, see below). "—" means not run. The default build's 1133 distinct solutions for `cent` are 404 correct plus 729 that do not evaluate to the target, matching `current.md`.

4.4 What this does and doesn't show

- Fixing defect 1 alone is enough for `cent_tiny` and `cent`; fixing defect 2 alone is enough for `nested` (Table 1). Defect 1 also fires in `nested` (Figure 5) but is harmless there once defect 2 is fixed, because the `FAIL_TAG` path re-saves `root`.
- Neither patch has been run against `tests/run.sh`, so this is evidence that the two defects are *sufficient* for these failures, not that scoped retry is now sound. I have not tried to construct a program where defect 1 pops a *choice* frame (the case noted in the caption of Figure 5), and I have not argued about ICaseLit or the free-variable path.
- Even with both patches, a retry in `CASE33` can never reach a choice made by `eval_hnf`: those frames are below `CASE33`'s cutoff (here `c3` has no alternative left anyway). Scoped retry stays a strictly local optimization; recovering such cases would need the cutoff to be inherited across `collapse`/`TAIL_DISPATCH` edges, and then the retry would also have to redo the *dispatch* that depended on the choice, not just re-read the scrutinee.
- `tests/run.sh --slow cent` currently reports an output mismatch for the default build: the golden file has no `SOLUTION: FAIL` lines, while every build I made, including scoped retry off, prints them (`nf_all` prints a solution unless `TAG(expr) == FAIL_TAG`; in the traces `expr` is a forward to a `FAIL` cell). This is independent of scoped retry and contradicts `current.md`'s note that the suite passes; I have not chased it further.